Programação Assíncrona em Python

Apostila de exercícios com diagramas de sequência

---

8 exercícios curtos para ficar fluente na sintaxe de `async`/`await`: cada um traz enunciado com lacunas, saída esperada, um **diagrama de sequência** do que acontece por baixo dos panos e, no final, o gabarito comentado.

O que você vai praticar

1. **Corrotinas, `await` e `asyncio.run`**
2. **Concorrência com `asyncio.gather`**
3. **Tarefas em segundo plano com `create_task`**
4. **Limites de tempo com `asyncio.timeout`**
5. **Produtor/consumidor com `asyncio.Queue`**
6. **Limitando concorrência com `Semaphore`**
7. **Erros e cancelamento: `gather` e `TaskGroup`**
8. **`async with` e `async for`**

Requisitos: Python 3.11 ou superior (usamos `asyncio.timeout`, `TaskGroup` e `except*`).

Sumário

Como usar esta apostila 2

Exercício 1 — Olá, corrotina 4

Exercício 2 — Várias coisas ao mesmo tempo 6

Exercício 3 — Tarefas em segundo plano 8

Exercício 4 — Limite de tempo 10

Exercício 5 — Produtor e consumidor 12

Exercício 6 — Limitando a concorrência 14

Exercício 7 — Erros e cancelamento 16

Exercício 8 — Context managers e iteradores assíncronos 18

Gabaritos 20

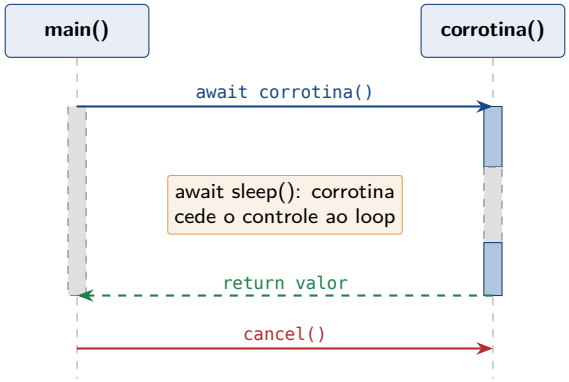
Como usar esta apostila

Fluxo sugerido. Para cada exercício: (1) leia o enunciado; (2) preencha os `TODO` no seu editor (os arquivos `exN_enunciado.py` acompanham a apostila); (3) rode com `python exN.py` e compare com a saída esperada; (4) só então olhe o diagrama e o gabarito.

Modelo mental em 4 linhas.

- Existe *um* event loop rodando em *uma* thread; ele executa uma corrotina por vez.
- A troca entre corrotinas acontece **somente** em um `await` (concorrência cooperativa).
- Chamar `f()` de uma função `async def` só cria o objeto corrotina; nada roda até haver `await` (ou uma Task).
- Código bloqueante (`time.sleep`, `requests.get`) trava o loop inteiro. Use as versões assíncronas.

Como ler os diagramas de sequência



- seta azul cheia: chamada, agendamento ou *retomada*
- seta verde tracejada: retorno de valor
- seta vermelha: exceção ou cancelamento
- barra azul: corrotina executando (ou dentro da seção protegida)
- barra cinza: corrotina suspensa, esperando algo
- nota: anotação explicativa

O tempo corre de cima para baixo. Rótulos como `t=1s` na margem esquerda marcam instantes reais. Nem todo diagrama mostra o Event Loop; quando ele não é relevante, foi omitido para ficar legível.

Cola de sintaxe

Construção	Para que serve
`async def f(): ...`	Define uma corrotina.
`await expr`	Suspende até `expr` terminar e devolve o resultado; cede o controle ao loop.
`asyncio.run(main())`	Cria o event loop, executa `main()` e fecha o loop. Ponto de entrada.
`await asyncio.sleep(s)`	Pausa cooperativa de `s` segundos (não bloqueia o loop).
`asyncio.create_task(c)`	Agenda `c` para rodar em segundo plano e devolve uma `Task`.
`await asyncio.gather(*cs)`	Roda várias corrotinas concorrentemente; devolve a lista de resultados.
`async with asyncio.timeout(s)`	Cancela o bloco se passar de `s` segundos (levanta `TimeoutError`).
`asyncio.Queue()`	Fila para comunicação entre corrotinas (`put`/`get` são `await`).
`asyncio.Semaphore(n)`	Permite no máximo `n` corrotinas ao mesmo tempo em uma seção.
`asyncio.TaskGroup()`	Concorrência estruturada: se uma tarefa falha, as irmãs são canceladas.
`async with` / `async for`	Gerenciadores de contexto e iteradores que podem fazer `await`.

Armadilhas clássicas

Esquecer o `await`: aparece `RuntimeWarning: coroutine ... was never awaited`. **Usar `time.sleep`:** congela todas as tarefas. **Chamar `asyncio.run` dentro de código já assíncrono:** só uma vez, no topo do programa.

Exercício 1 — Olá, corrotina

Conceitos: `async def`, `await`, `asyncio.run`, `asyncio.sleep`

Complete a corrotina `saudacao` e a função `main`. Em `main`, chame `saudacao("Ana", 1)` **e depois** `saudacao("Beto", 2)`, usando `await` em cada uma, e meça o tempo total. Repare que o total é $1+2=3$ segundos: um `await` direto *não* cria concorrência. Isso prepara o terreno para o exercício 2.

Código-base

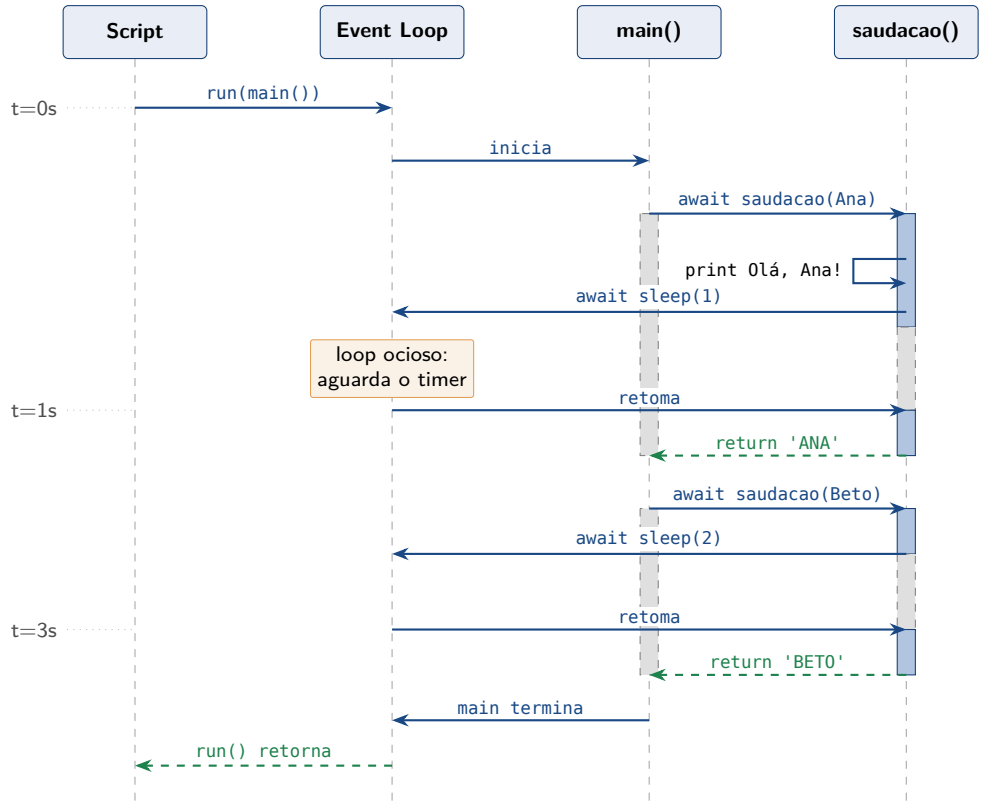
```python
import asyncio
import time


async def saudacao(nome: str, atraso: float) -> str:
    # TODO 1: imprima "Olá, {nome}!"
    # TODO 2: aguarde `atraso` segundos SEM bloquear o event loop
    # TODO 3: imprima "Tchau, {nome}!" e retorne o nome em maiúsculas
    ...


async def main():
    inicio = time.perf_counter()
    # TODO 4: chame saudacao("Ana", 1) e depois saudacao("Beto", 2),
    #         guardando os retornos em r1 e r2
    ...
    print(f"Retornos: {r1}, {r2}")
    print(f"Tempo total: {time.perf_counter() - inicio:.1f}s")


# TODO 5: inicie o event loop executando main()
```

Diagrama de sequência



Saída esperada

```
Olá, Ana!
Tchau, Ana!
Olá, Beto!
Tchau, Beto!
Retornos: ANA, BETO
Tempo total: 3.0s
```

Dica

`asyncio.run(main())` *chama* `main()` (isso cria a corrotina) e entrega o objeto ao loop; não escreva `asyncio.run(main)` sem os parênteses.

Exercício 2 — Várias coisas ao mesmo tempo

Conceitos: `asyncio.gather`, concorrência vs. sequência

Três "downloads" simulados levam 3 s, 1 s e 2 s. Use `asyncio.gather` para executá-los **concorrentemente** e imprima a lista de resultados. O tempo total deve ser o do mais lento (3 s), não a soma (6 s). Observe duas ordens diferentes: as linhas `concluído` saem na ordem de *término*, mas a lista devolvida segue a ordem dos *argumentos*.

Código-base

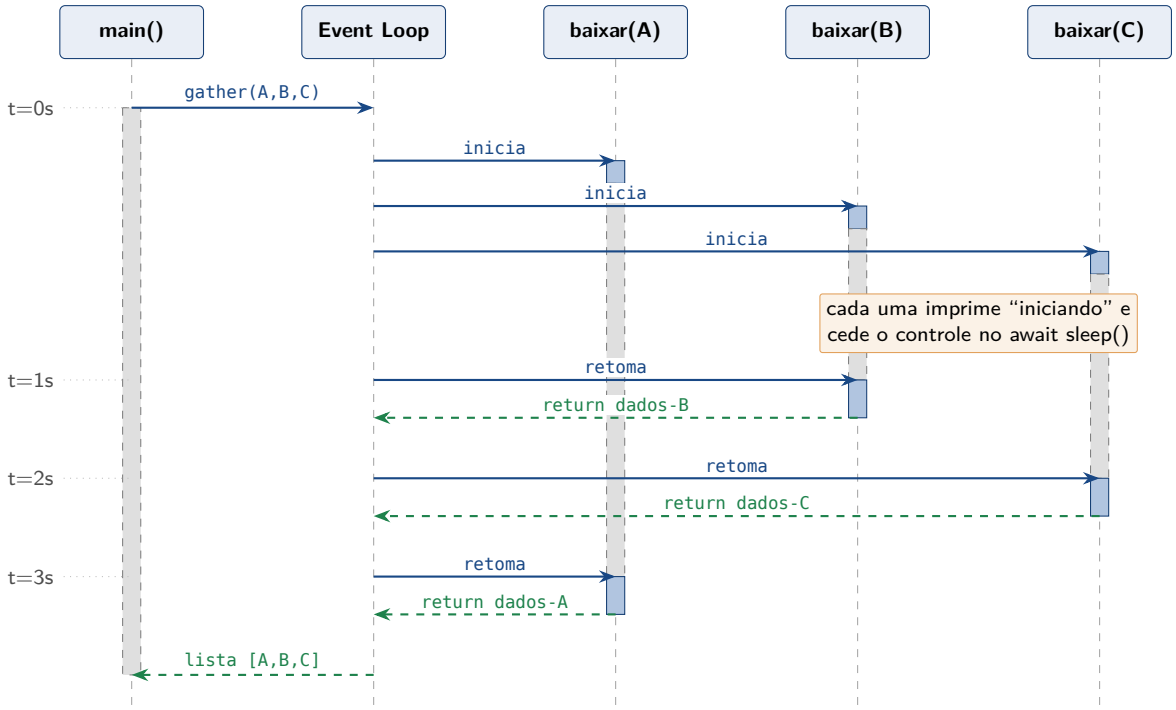
```python
import asyncio
import time


async def baixar(nome: str, atraso: float) -> str:
    print(f"[{nome}] iniciando")
    await asyncio.sleep(atraso)
    print(f"[{nome}] concluído")
    return f"dados-{nome}"


async def main():
    inicio = time.perf_counter()
    # TODO: rode baixar("A", 3), baixar("B", 1) e baixar("C", 2)
    #       CONCORRENTEMENTE com asyncio.gather e guarde a lista
    resultados = ...
    print(resultados)
    print(f"Tempo: {time.perf_counter() - inicio:.1f}s")


asyncio.run(main())
```

Diagrama de sequência



Saída esperada

```
[A] iniciando
[B] iniciando
[C] iniciando
[B] concluído
[C] concluído
[A] concluído
['dados-A', 'dados-B', 'dados-C']
Tempo: 3.0s
```

Dica

`gather` aceita corrotinas *soltas*, não uma lista: para uma lista `xs`, use `await asyncio.gather(*xs)`.

Exercício 3 — Tarefas em segundo plano

Conceitos: `asyncio.create_task`, agendar vs. aguardar

`create_task` *agenda* uma corrotina e devolve imediatamente uma `Task`; ela só começa a rodar quando quem agendou fizer um `await`. Agende `trabalhar("fundo", 3)`, deixe `main` fazer outras coisas por 1,5 s e só depois aguarde a task. Note que a task avança *enquanto* `main` está dormindo.

Código-base

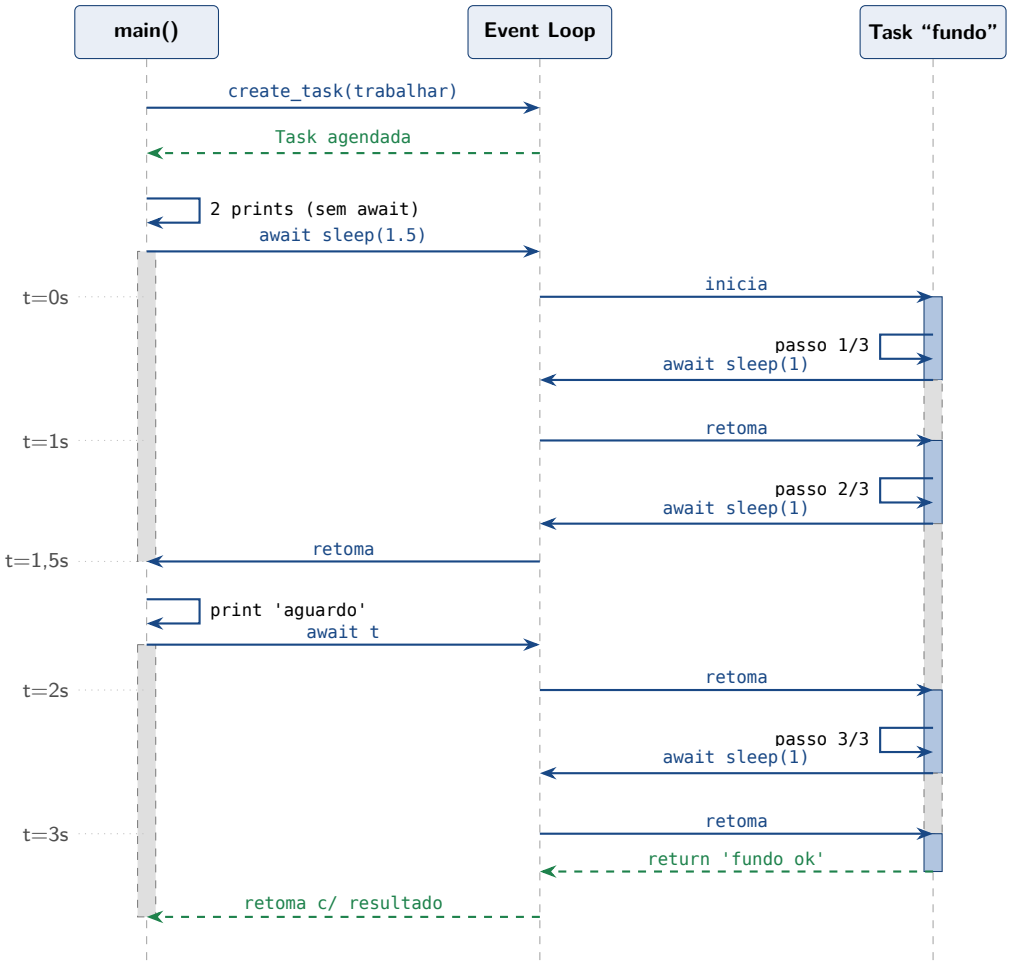
```python
import asyncio


async def trabalhar(nome: str, passos: int) -> str:
    for i in range(1, passos + 1):
        print(f"  [{nome}] passo {i}/{passos}")
        await asyncio.sleep(1)
    return f"{nome} ok"


async def main():
    # TODO 1: agende trabalhar("fundo", 3) com asyncio.create_task
    #         (sem usar await agora!) e guarde em t
    t = ...
    print("main: tarefa agendada (ainda não começou)")
    print("main: fazendo outra coisa por 1.5s...")
    await asyncio.sleep(1.5)
    print("main: agora aguardo a tarefa")
    # TODO 2: aguarde a task terminar e guarde o retorno em resultado
    resultado = ...
    print("main: resultado =", resultado)


asyncio.run(main())
```

Diagrama de sequência



Saída esperada

```
main: tarefa agendada (ainda não começou)
main: fazendo outra coisa por 1.5s...
  [fundo] passo 1/3
  [fundo] passo 2/3
main: agora aguardo a tarefa
  [fundo] passo 3/3
main: resultado = fundo ok
```

Dica

Se você escrever `await trabalhar(...)` no lugar de `create_task`, tudo vira sequencial e a ordem das linhas muda. Compare! Se o programa terminar sem aguardar a task, ela é descartada.

Exercício 4 — Limite de tempo

Conceitos: `asyncio.timeout`, `CancelledError`, `TimeoutError`

Uma consulta pode demorar mais que o tolerável. Envolva a chamada a `consulta_lenta` em `async with asyncio.timeout(2)`. A consulta de 1 s deve funcionar; a de 5 s deve ser *cancelada* e `main` deve receber `TimeoutError`. Repare que a própria corrotina vê um `CancelledError` e deve relançá-lo.

Código-base

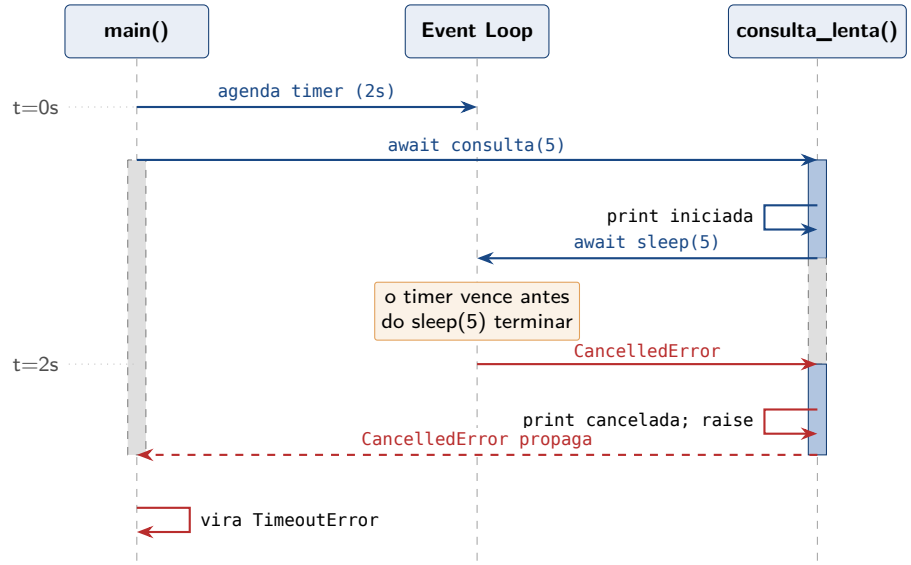
```python
import asyncio


async def consulta_lenta(segundos: float) -> str:
    print(f"consulta: iniciada ({segundos}s)")
    try:
        await asyncio.sleep(segundos)
        return "dados"
    except asyncio.CancelledError:
        print("consulta: cancelada!")
        raise                       # sempre re-lance CancelledError


async def main():
    for duracao in (1, 5):
        try:
            # TODO: limite a chamada abaixo a 2 segundos usando
            #       "async with asyncio.timeout(...)"
            r = await consulta_lenta(duracao)
            print(f"consulta de {duracao}s: {r}")
        except TimeoutError:
            print(f"consulta de {duracao}s: estourou o tempo (2s)")


asyncio.run(main())
```

Diagrama de sequência (consulta de 5 s)



Saída esperada

```
consulta: iniciada (1s)
consulta de 1s: dados
consulta: iniciada (5s)
consulta: cancelada!
consulta de 5s: estourou o tempo (2s)
```

Dica

Antes do Python 3.11 o equivalente era `await asyncio.wait_for(consulta_lenta(5), timeout=2)`. Nunca engula o `CancelledError` sem relançá-lo.

Exercício 5 — Produtor e consumidor

Conceitos: `asyncio.Queue`, `put`/`get`, sentinela de término

O produtor gera 3 itens (um a cada 0,4 s) e o consumidor leva 1 s para processar cada um; portanto a fila acumula itens. Complete `put`, `get` e a criação da fila em `main`. O produtor sinaliza o fim enviando `None`.

Código-base

```python
import asyncio

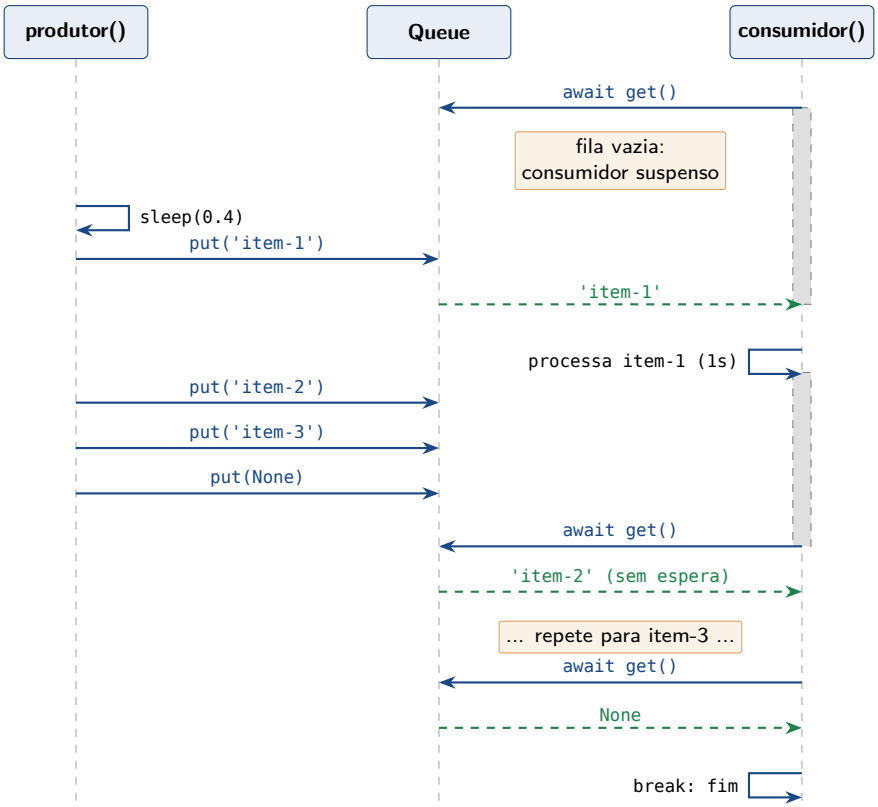

async def produtor(fila: asyncio.Queue, n: int):
    for i in range(1, n + 1):
        await asyncio.sleep(0.4)
        item = f"item-{i}"
        # TODO 1: coloque `item` na fila (await!)
        print(f"produtor: colocou {item}")
    # TODO 2: coloque None na fila como sentinela de "fim"


async def consumidor(fila: asyncio.Queue):
    while True:
        # TODO 3: retire um item da fila (await!)
        item = ...
        if item is None:
            break
        print(f"consumidor: processando {item}")
        await asyncio.sleep(1)
        print(f"consumidor: terminou {item}")


async def main():
    # TODO 4: crie a fila e rode produtor(fila, 3) e consumidor(fila)
    #         ao mesmo tempo
    ...


asyncio.run(main())
```

Diagrama de sequência



Saída esperada

```
produtor: colocou item-1
consumidor: processando item-1
produtor: colocou item-2
produtor: colocou item-3
consumidor: terminou item-1
consumidor: processando item-2
consumidor: terminou item-2
consumidor: processando item-3
consumidor: terminou item-3
```

Dica

`Queue(maxsize=2)` faz o `put` *suspender* quando a fila está cheia (contrapressão). Desafio extra: teste isso e observe como a ordem das linhas muda.

Exercício 6 — Limitando a concorrência

Conceitos: `asyncio.Semaphore`, `async with`

Cinco workers disputam um recurso que só aceita 2 por vez (pense num limite de requisições simultâneas a uma API). Use um `Semaphore(2)` com `async with sem:` para garantir que nunca haja mais de 2 workers dentro da seção protegida. Como cada um leva 1 s, o total esperado é $\lceil 5/2 \rceil = 3$ s.

Código-base

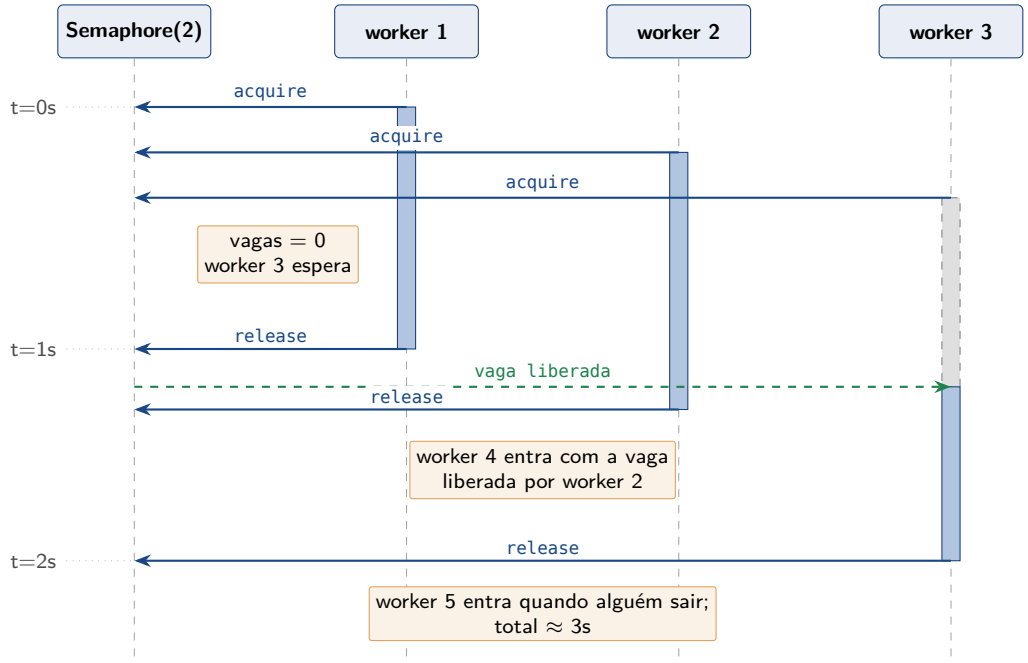
```python
import asyncio
import time


async def worker(i: int, sem: asyncio.Semaphore):
    # TODO 1: proteja o bloco abaixo com "async with sem:"
    print(f"worker {i}: entrou")
    await asyncio.sleep(1)
    print(f"worker {i}: saiu")


async def main():
    inicio = time.perf_counter()
    # TODO 2: crie um Semaphore que permita só 2 workers simultâneos
    sem = ...
    # TODO 3: dispare 5 workers (i = 1..5) com gather
    #         dica: gather(*(... for i in range(1, 6)))
    ...
    print(f"Tempo total: {time.perf_counter() - inicio:.1f}s")


asyncio.run(main())
```

Diagrama de sequência (3 dos 5 workers)



Saída esperada

```
worker 1: entrou
worker 2: entrou
worker 1: saiu
worker 2: saiu
worker 3: entrou
worker 4: entrou
worker 3: saiu
worker 4: saiu
worker 5: entrou
worker 5: saiu
Tempo total: 3.0s
```

Dica

Linhas que ocorrem no *mesmo instante* podem aparecer em ordem levemente diferente na sua máquina; o que não muda é que nunca há mais de 2 `entrou` sem um `saiu` correspondente.

Exercício 7 — Erros e cancelamento

Conceitos: `gather(return_exceptions=True)`, `TaskGroup`, `except*`

Parte A: rode `ok("A",1)`, `falha(2)` e `ok("C",3)` com `gather` sem deixar a exceção derrubar as demais (`return_exceptions=True`); a exceção aparece *como item* da lista.
Parte B: rode `ok("D",1)`, `falha(2)` e `lenta()` dentro de um `TaskGroup`. Quando `falha` levanta o erro, o grupo **cancela as irmãs** e `main` captura tudo com `except* ValueError`.

Código-base

```python
import asyncio


async def ok(nome: str, atraso: float) -> str:
    await asyncio.sleep(atraso)
    print(f"{nome}: ok")
    return nome


async def falha(atraso: float):
    await asyncio.sleep(atraso)
    raise ValueError("deu ruim!")

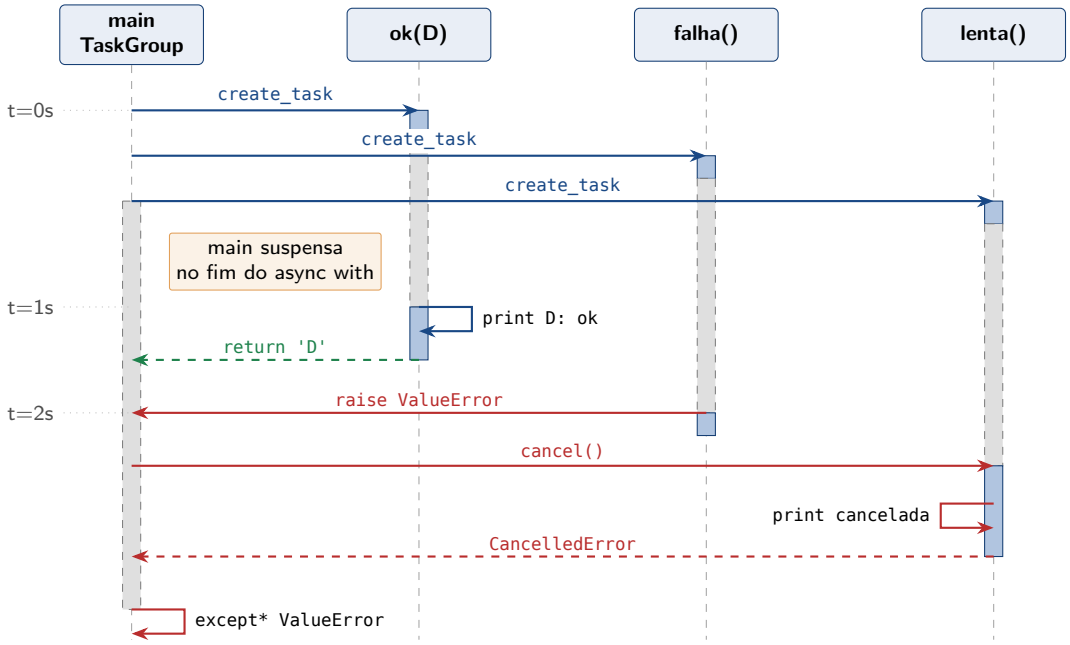

async def lenta():
    try:
        await asyncio.sleep(10)
    except asyncio.CancelledError:
        print("lenta: cancelada")
        raise


async def main():
    # Parte A: rode ok("A",1), falha(2), ok("C",3) com gather,
    # SEM deixar a exceção derrubar tudo (return_exceptions=True)
    resultados = ...
    for r in resultados:
        print(repr(r))

    # Parte B: rode ok("D",1), falha(2) e lenta() dentro de um
    # asyncio.TaskGroup e capture o erro com "except* ValueError"
    print("--- TaskGroup ---")
    ...


asyncio.run(main())
```

Diagrama de sequência (Parte B)



Saída esperada

```
A: ok
C: ok
'A'
ValueError('deu ruim!')
'C'
--- TaskGroup ---
D: ok
lenta: cancelada
capturado: (ValueError('deu ruim!'),)
```

Dica

`except*` captura um `ExceptionGroup`; `eg.exceptions` lista os erros individuais. Sem `return_exceptions=True`, o `gather` propaga a primeira exceção, mas **não** cancela as outras tarefas, o que o `TaskGroup` faz por você.

Exercício 8 — Context managers e iteradores assíncronos

Conceitos: `async with`, `__aenter__`/`__aexit__`, `async for`, gerador assíncrono

Implemente uma classe `Conexao` cujo `__aenter__` e `__aexit__` fazem `await` (abrir e fechar levam tempo), e um **gerador assíncrono** `contador` (uma função `async def` com `yield`). Em `main`, abra a conexão com `async with` e consuma o contador com `async for`.

Código-base

```python
import asyncio


class Conexao:
    def __init__(self, nome: str):
        self.nome = nome

    # TODO 1: implemente __aenter__ (async!):
    #         imprima "{nome}: abrindo...", espere 0.5s,
    #         imprima "{nome}: aberta" e retorne self

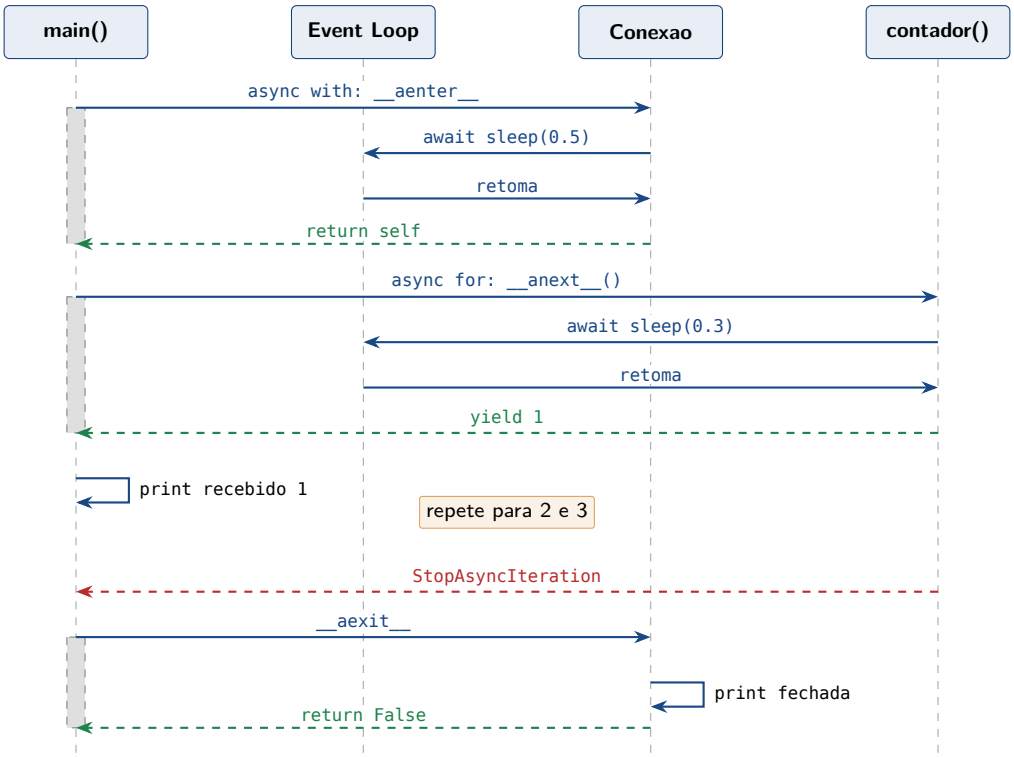
    # TODO 2: implemente __aexit__(self, exc_type, exc, tb) (async!):
    #         espere 0.2s, imprima "{nome}: fechada" e retorne False


# TODO 3: escreva o gerador assíncrono contador(ate, atraso):
#         para i de 1 até `ate`: espere `atraso` e faça yield i


async def main():
    # TODO 4: abra a Conexao("db") com "async with" ...
    #         TODO 5: ... e consuma contador(3, 0.3) com "async for",
    #                 imprimindo "recebido {valor}"
    ...


asyncio.run(main())
```

Diagrama de sequência



Saída esperada

```
db: abrindo...
db: aberta
recebido 1
recebido 2
recebido 3
db: fechada
```

Dica

`StopAsyncIteration` é o sinal normal de fim do laço `async for`, não um erro. Bônus: reescreva o consumo como `valores = [v async for v in contador(3, 0.1)]`.

Gabaritos

Cada solução abaixo foi executada para gerar as saídas mostradas nos exercícios. Se a sua versão difere mas produz o mesmo comportamento, está ótimo.

Exercício 1

```python
import asyncio
import time


async def saudacao(nome: str, atraso: float) -> str:
    print(f"Olá, {nome}!")
    await asyncio.sleep(atraso)        # cede o controle ao event loop
    print(f"Tchau, {nome}!")
    return nome.upper()


async def main():
    inicio = time.perf_counter()
    r1 = await saudacao("Ana", 1)      # só segue quando terminar
    r2 = await saudacao("Beto", 2)
    print(f"Retornos: {r1}, {r2}")
    print(f"Tempo total: {time.perf_counter() - inicio:.1f}s")


asyncio.run(main())
```

Exercício 2

```python
import asyncio
import time


async def baixar(nome: str, atraso: float) -> str:
    print(f"[{nome}] iniciando")
    await asyncio.sleep(atraso)
    print(f"[{nome}] concluído")
    return f"dados-{nome}"


async def main():
    inicio = time.perf_counter()
    resultados = await asyncio.gather(
        baixar("A", 3),
        baixar("B", 1),
        baixar("C", 2),
    )
    print(resultados)
    print(f"Tempo: {time.perf_counter() - inicio:.1f}s")


asyncio.run(main())
```

Exercício 3

```python
import asyncio


async def trabalhar(nome: str, passos: int) -> str:
    for i in range(1, passos + 1):
        print(f"  [{nome}] passo {i}/{passos}")
        await asyncio.sleep(1)
    return f"{nome} ok"


async def main():
    t = asyncio.create_task(trabalhar("fundo", 3))   # agenda, não espera
    print("main: tarefa agendada (ainda não começou)")
    print("main: fazendo outra coisa por 1.5s...")
    await asyncio.sleep(1.5)                         # aqui a task roda
    print("main: agora aguardo a tarefa")
    resultado = await t
    print("main: resultado =", resultado)


asyncio.run(main())
```

Exercício 4

```python
import asyncio


async def consulta_lenta(segundos: float) -> str:
    print(f"consulta: iniciada ({segundos}s)")
    try:
        await asyncio.sleep(segundos)
        return "dados"
    except asyncio.CancelledError:
        print("consulta: cancelada!")
        raise                       # sempre re-lance CancelledError


async def main():
    for duracao in (1, 5):
        try:
            async with asyncio.timeout(2):          # Python 3.11+
                r = await consulta_lenta(duracao)
            print(f"consulta de {duracao}s: {r}")
        except TimeoutError:
            print(f"consulta de {duracao}s: estourou o tempo (2s)")


asyncio.run(main())
```

Exercício 5

```python
import asyncio


async def produtor(fila: asyncio.Queue, n: int):
    for i in range(1, n + 1):
        await asyncio.sleep(0.4)
        item = f"item-{i}"
        await fila.put(item)
        print(f"produtor: colocou {item}")
    await fila.put(None)                # sentinela: "acabou"


async def consumidor(fila: asyncio.Queue):
    while True:
        item = await fila.get()         # suspende se a fila estiver vazia
        if item is None:
            break
        print(f"consumidor: processando {item}")
        await asyncio.sleep(1)
        print(f"consumidor: terminou {item}")


async def main():
    fila = asyncio.Queue()
    await asyncio.gather(produtor(fila, 3), consumidor(fila))


asyncio.run(main())
```

Exercício 6

```python
import asyncio
import time


async def worker(i: int, sem: asyncio.Semaphore):
    async with sem:                     # acquire / release automáticos
        print(f"worker {i}: entrou")
        await asyncio.sleep(1)
        print(f"worker {i}: saiu")


async def main():
    inicio = time.perf_counter()
    sem = asyncio.Semaphore(2)          # no máximo 2 ao mesmo tempo
    await asyncio.gather(*(worker(i, sem) for i in range(1, 6)))
    print(f"Tempo total: {time.perf_counter() - inicio:.1f}s")


asyncio.run(main())
```

Exercício 7

```python
import asyncio


async def ok(nome: str, atraso: float) -> str:
    await asyncio.sleep(atraso)
    print(f"{nome}: ok")
    return nome


async def falha(atraso: float):
    await asyncio.sleep(atraso)
    raise ValueError("deu ruim!")


async def lenta():
    try:
        await asyncio.sleep(10)
    except asyncio.CancelledError:
        print("lenta: cancelada")
        raise


async def main():
    # Parte A: gather coleta as exceções como resultados
    resultados = await asyncio.gather(
        ok("A", 1), falha(2), ok("C", 3),
        return_exceptions=True,
    )
    for r in resultados:
        print(repr(r))

    # Parte B: TaskGroup cancela as irmãs quando uma falha
    print("--- TaskGroup ---")
    try:
        async with asyncio.TaskGroup() as tg:
            tg.create_task(ok("D", 1))
            tg.create_task(falha(2))
            tg.create_task(lenta())
    except* ValueError as eg:
        print("capturado:", eg.exceptions)


asyncio.run(main())
```

Exercício 8

```python
import asyncio


class Conexao:
    def __init__(self, nome: str):
        self.nome = nome

    async def __aenter__(self):
        print(f"{self.nome}: abrindo...")
        await asyncio.sleep(0.5)
        print(f"{self.nome}: aberta")
        return self

    async def __aexit__(self, exc_type, exc, tb):
        await asyncio.sleep(0.2)
        print(f"{self.nome}: fechada")
        return False                        # não suprime exceções


async def contador(ate: int, atraso: float):
    """Gerador assíncrono: produz 1..ate, esperando entre os valores."""
    for i in range(1, ate + 1):
        await asyncio.sleep(atraso)
        yield i


async def main():
    async with Conexao("db") as con:
        async for valor in contador(3, 0.3):
            print("recebido", valor)


asyncio.run(main())
```

Próximos passos

Troque `asyncio.sleep` por I/O real: `aiohttp` ou `httpx.AsyncClient` para HTTP, `asyncpg` para PostgreSQL, `aiofiles` para arquivos. Para código bloqueante que não tem versão assíncrona, use `await asyncio.to_thread(func, ...)`.